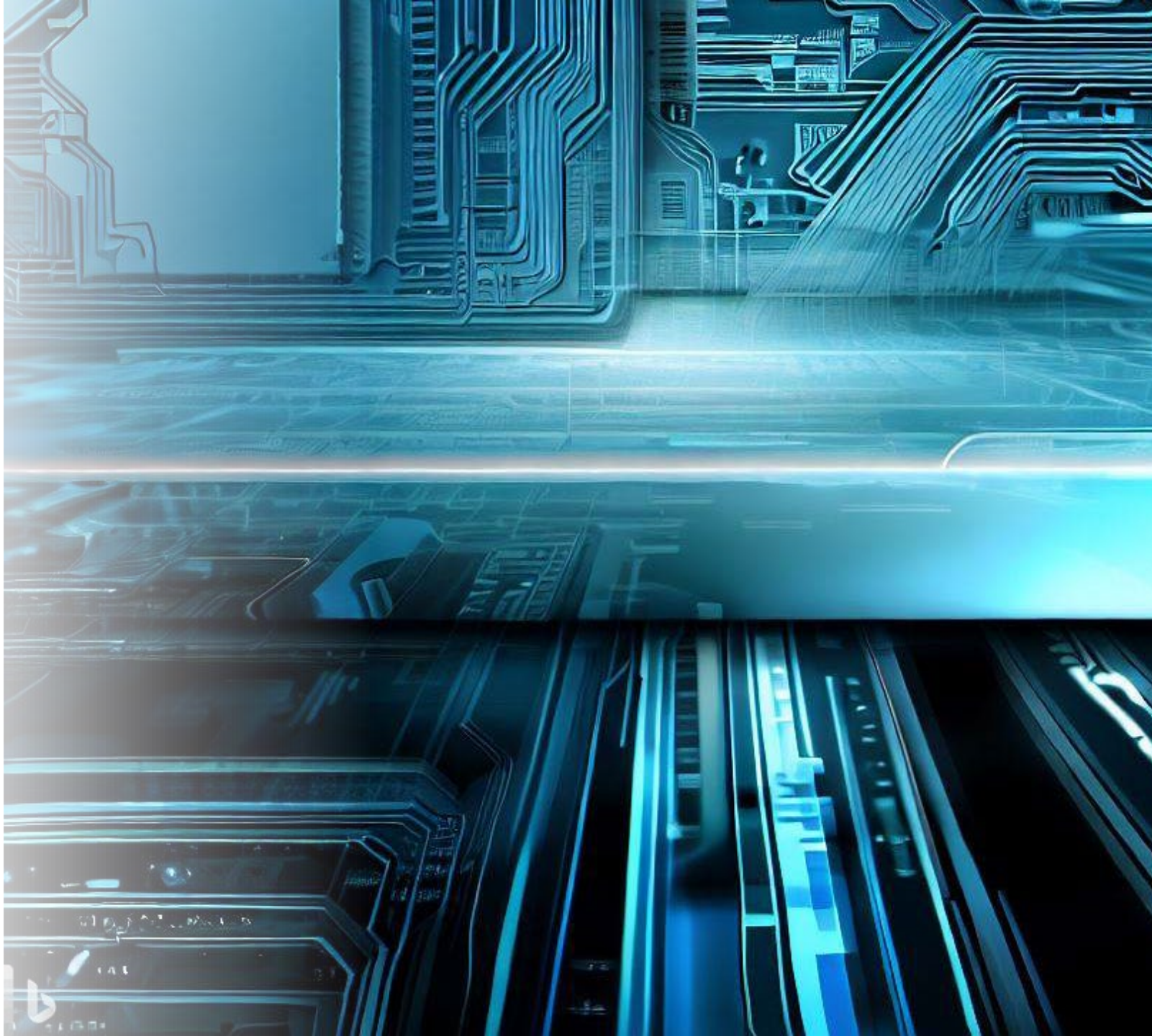


SC1006

Computer Organisation and Architecture

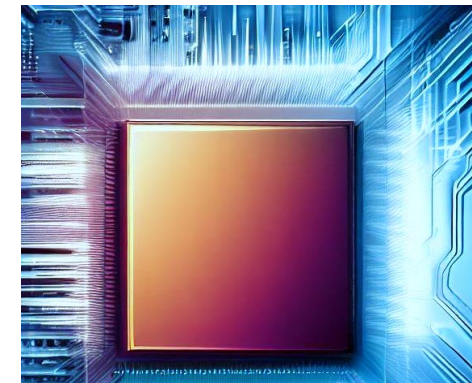
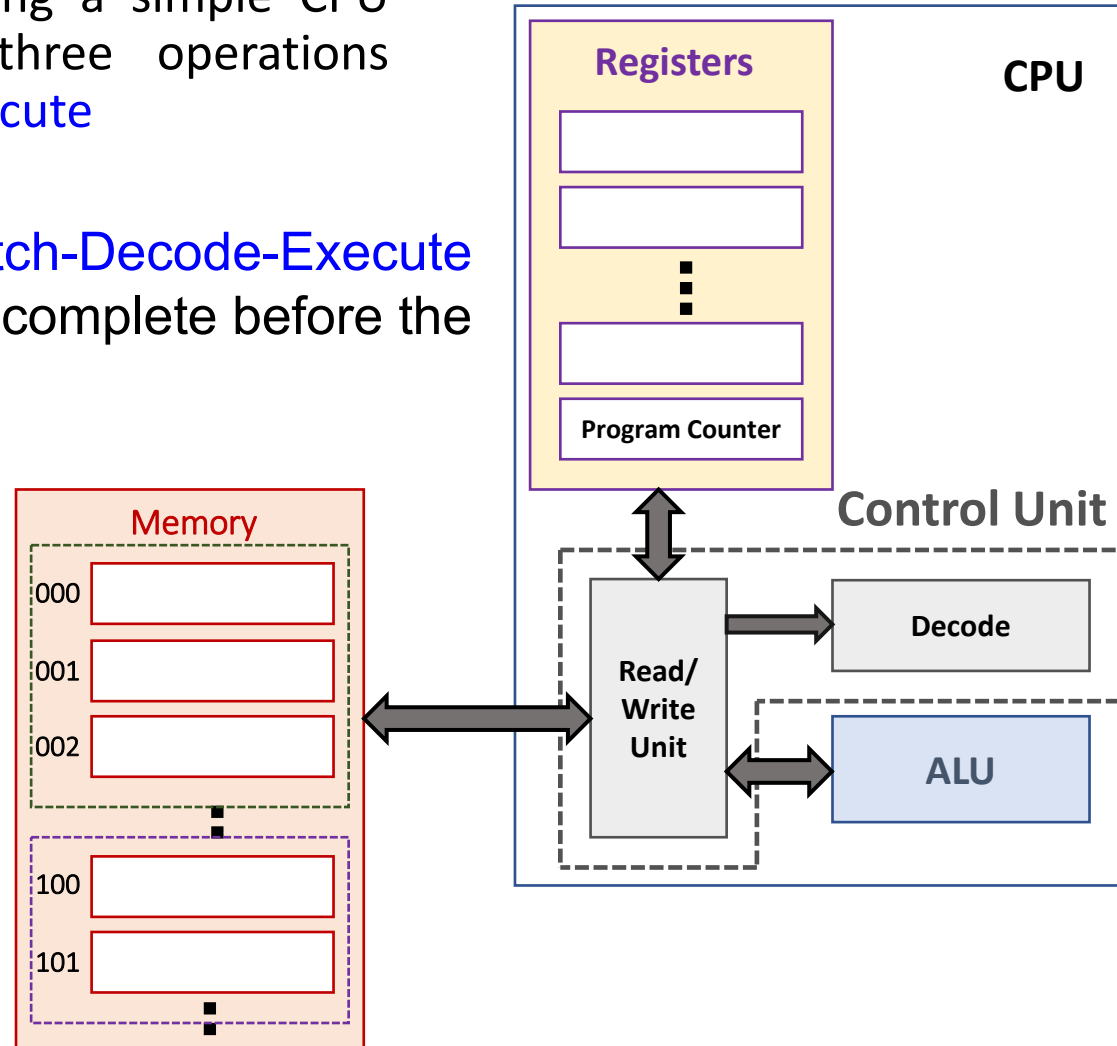
Processor Pipeline

Prof Lin WeiSi
College of Computing and
Data Science, NTU
Email: wslin@ntu.edu.sg

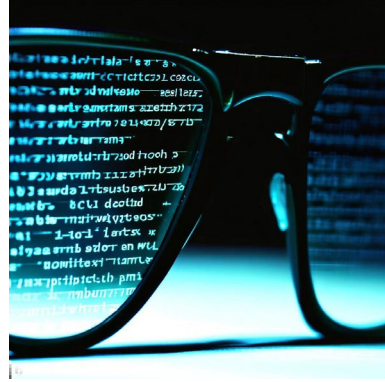


Review of a Simple CPU

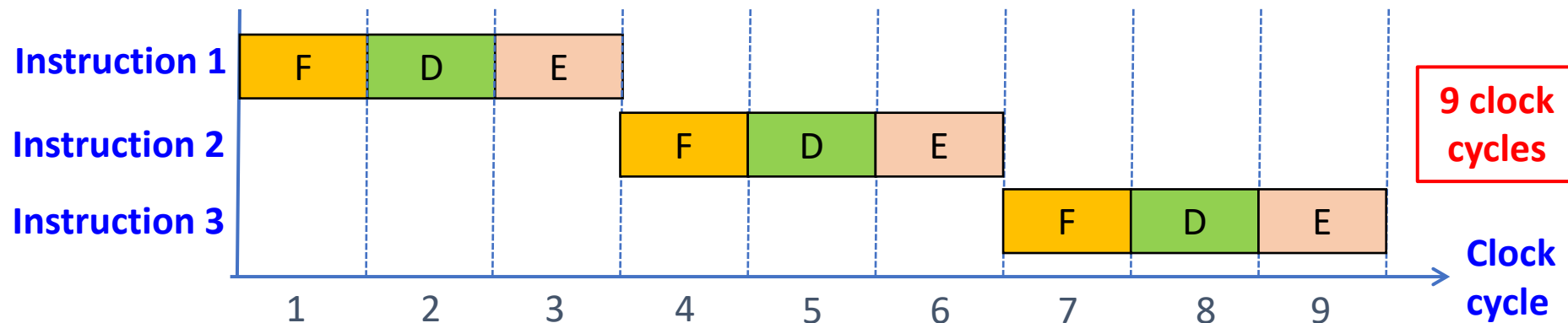
- So far we have been discussing a simple CPU which does the following three operations **sequentially**: **Fetch-Decode-Execute**
- In the simple CPU, the **Fetch-Decode-Execute** cycle of an instruction must complete before the next instruction is fetched



Instruction Execution – Simple CPU

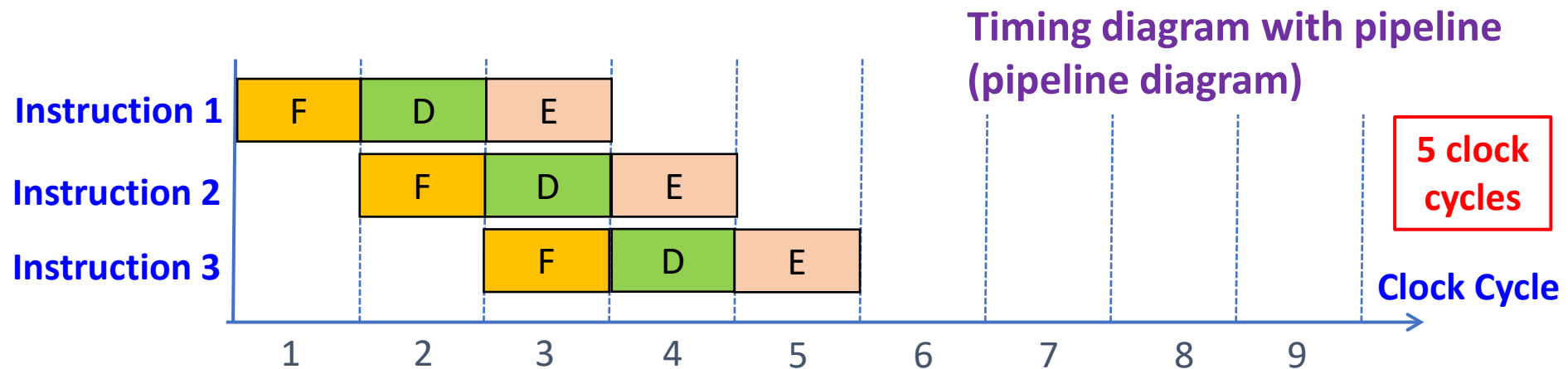
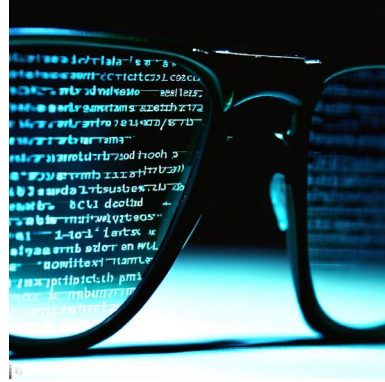


- Consider the simple CPU discussed in previous slide.
- Each instruction is broken down into the following stages
 - Fetch Instruction (F),
 - Decode (D)
 - Execute (E)
- Executing each instruction is equivalent to cycling through the three stages F, D and E. If we assume that each stage takes one clock cycle, the time taken to execute 3 instruction will be 9 cycles.



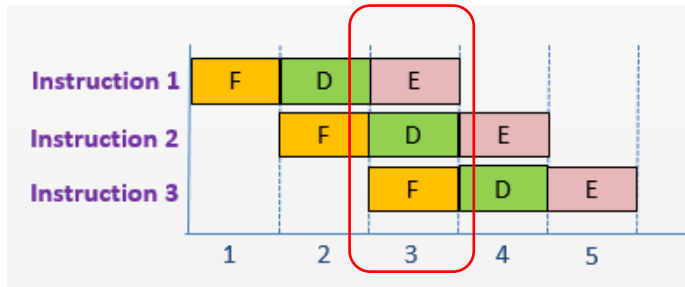
Pipeline Architecture

- This is an example of a Pipeline Processor Architecture.
- Pipelining is about **partitioning an instruction into simpler stages and assigning resources to allow these stages to be executed simultaneously.**
- There are many ways to partition an instruction, the example shown here is one of the simplest. Its not uncommon for more complex application processors to have more than 10 pipeline stages.



Pipeline – A peek into Processor Internal

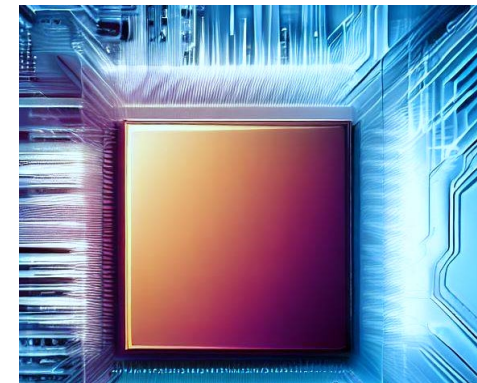
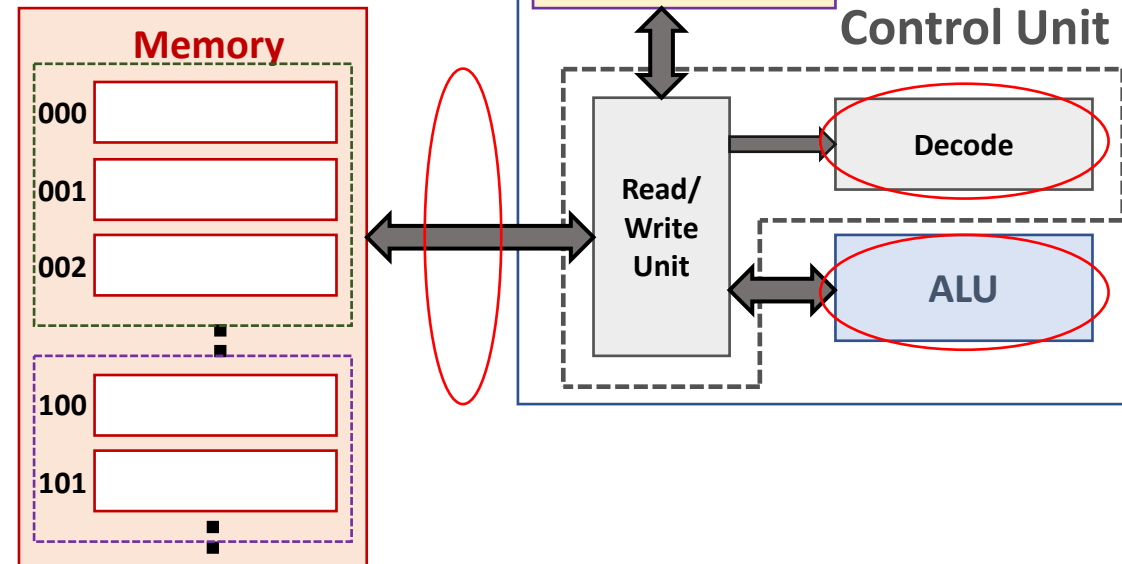
- Pipelining is possible if the Fetch-Decode-Execute operations use **independent resources**
- For example
 - Fetch (External buses)
 - Decode (Instruction Decoder)
 - Execute (ALU)



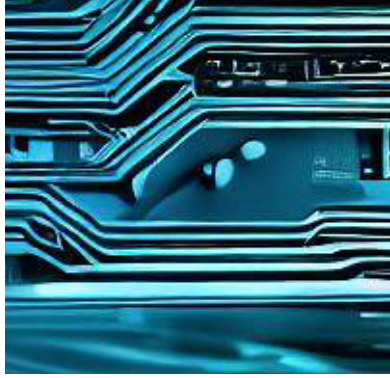
Fetch Instruction

Decode Instruction

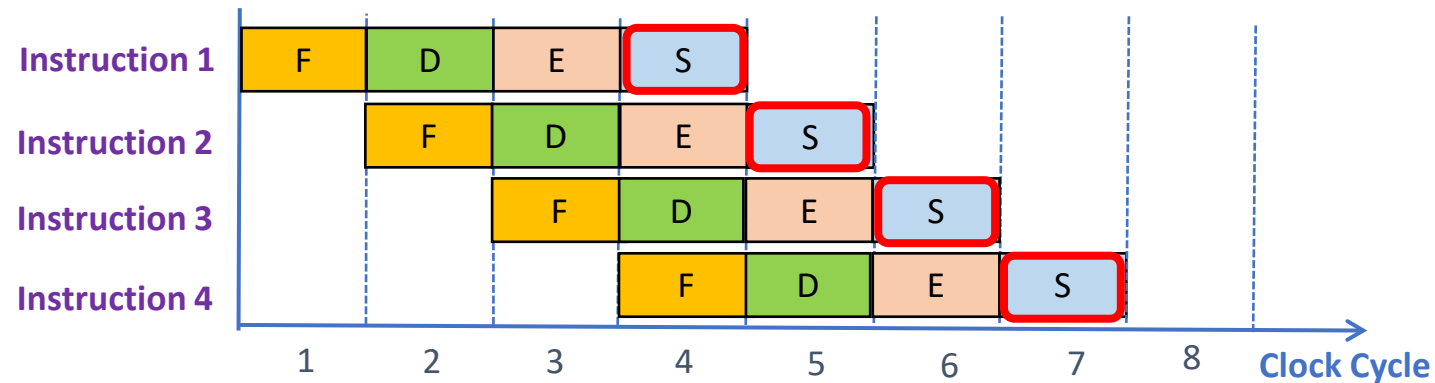
Execute Instruction



Pipeline Efficiency

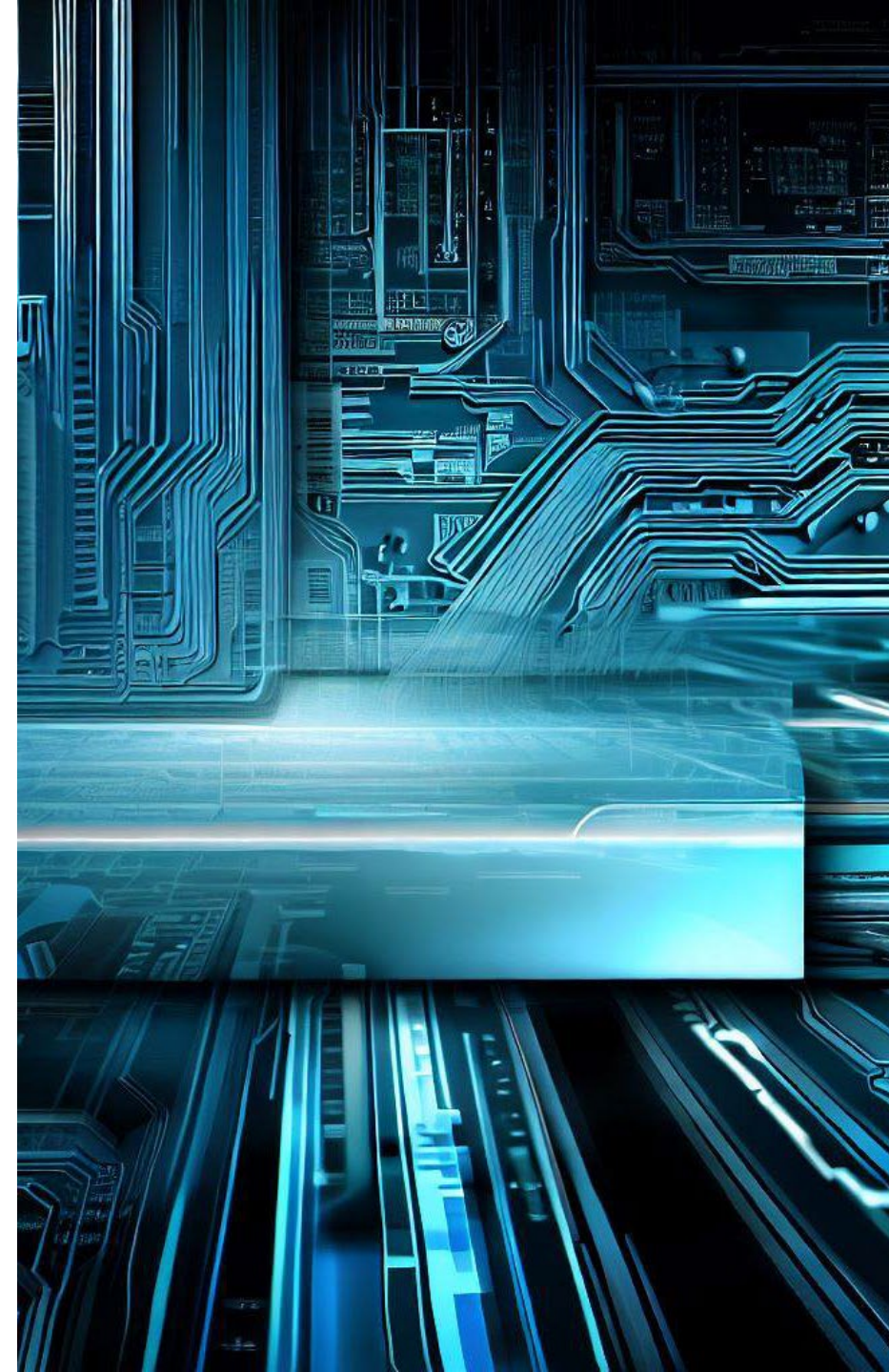


- The **efficiency** of a Pipeline architecture is **maximised** when the **pipeline is filled**
- Supposed we have a 4-stage pipeline processor
 - Instruction Fetch (F)
 - Instruction Decode (D)
 - Instruction Execution (E)
 - Data Store (S)
- It takes **4 cycles to fill the pipeline** and release the first instruction from the pipeline.
- But after the pipeline is filled, it is able to release **one instruction every cycle**, giving the effect of 'executing' one instruction every clock cycle.



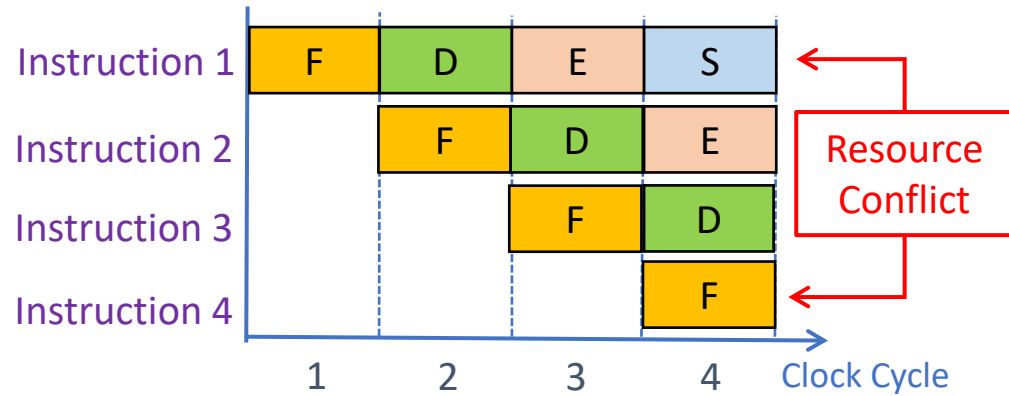
Pipeline Conflicts

- Pipeline efficiency mentioned in the previous slide will be reduced drastically if there are disruption to the pipeline.
- Events that disrupt the pipeline is known as pipeline conflicts.
- Pipeline conflicts typically cause a temporary halt to the pipeline or in some cases, the processor has to flush the instructions in the pipeline and reload with a fresh set of instructions.
- These actions result in additional time to execute a particular set of instructions.
- Since the total time taken to execute the instruction is longer, the number of instructions that can be released from the pipeline is reduced, which means the effective performance of the processor is lowered.
- In this module, we will look at three sources of pipeline conflict
 - Insufficient Resource
 - Data Dependency between instructions
 - Pipeline flushing due to Branch Instruction



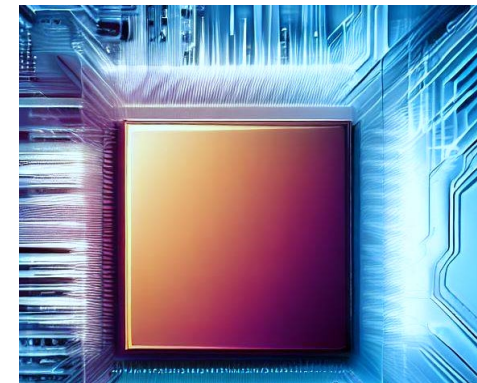
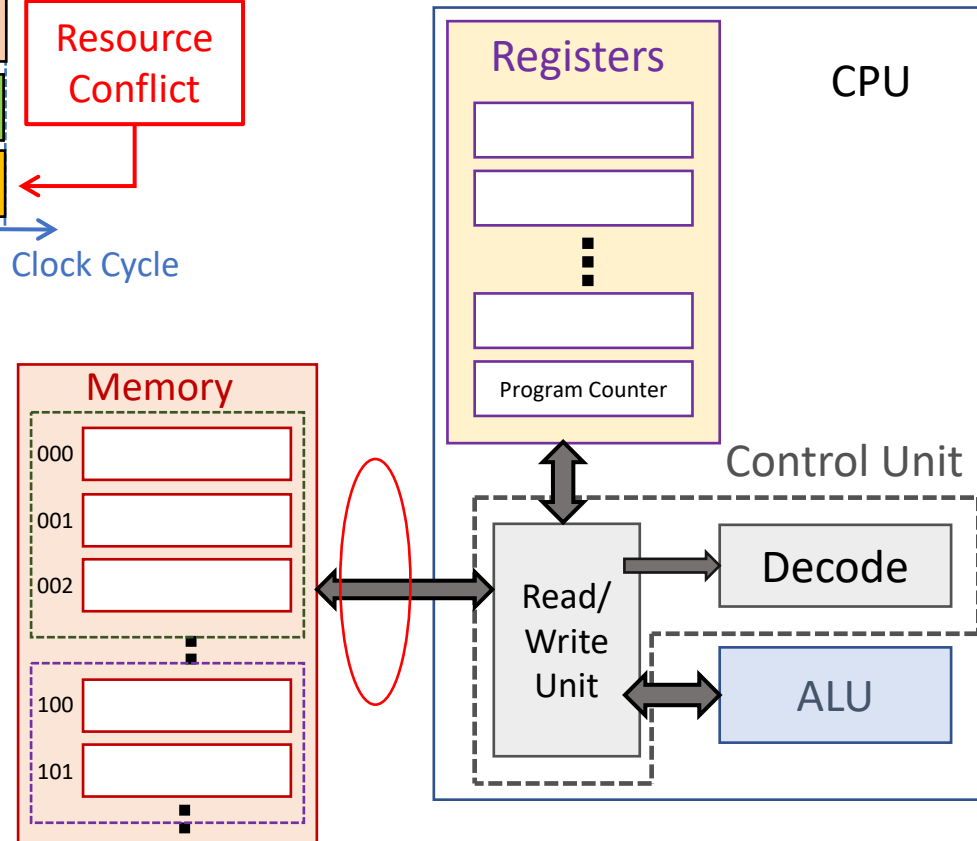
Resource Conflict

- Consider a processor with 4 pipeline stages: Fetch Instruction (F), Decode (D), Execute (E), Store Result (S)



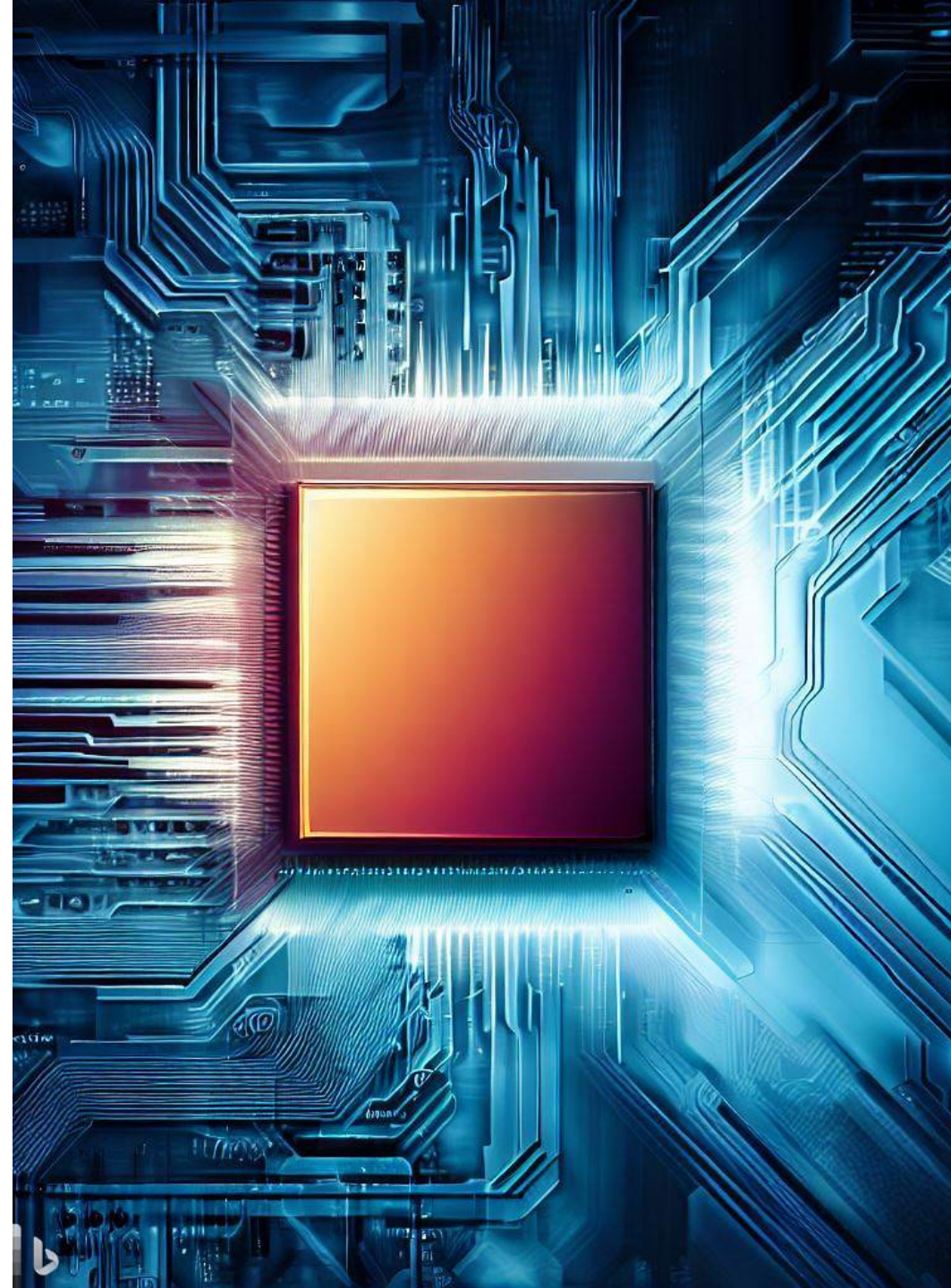
Example: Instruction 1 is storing result in memory when instruction 4 needs to fetch a new instruction

- Resource conflict occurs when two instructions attempt to access the same resource in the same cycle.



Resolving Resource Conflicts

- You need sufficient resources for a pipeline processor to work efficiently
- A pipeline processor with insufficient resources to operate each pipeline stage simultaneously will result in constant halting and flushing of pipeline.
- It's common for pipeline processors to have multiple resources: internal buses (data, instruction, peripherals), control and processing units etc to allow simultaneous operations in any instance.



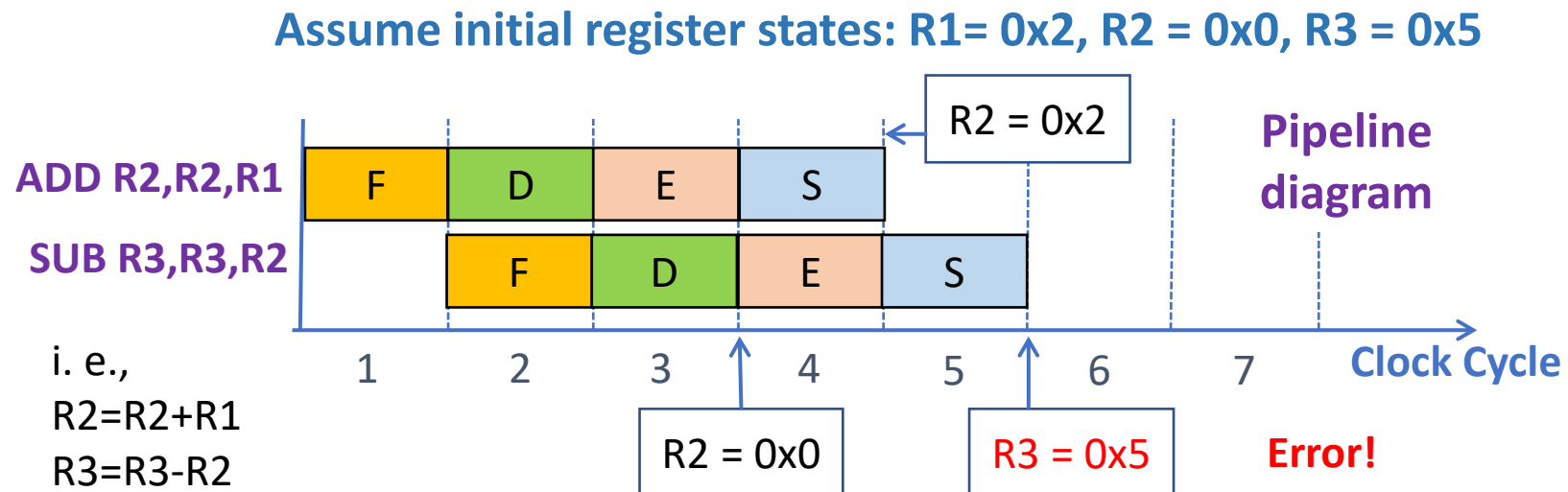
Use of ARM instructions in Pipeline Examples

- ARM instructions are used in the Pipeline examples but note that the following key differences
 - The pipeline structure discussed is **NOT** that of the ARM pipeline
 - For simplification of analysis, **ALL** the ARM instructions used are **assumed to occupy only one word** and **each pipeline stage take one clock cycle** to execute.
 - The suffix 'S' feature is enabled by default, i.e. all instructions will affect the conditional flag. E.g. ADD = ADDS, MOV = MOVS.



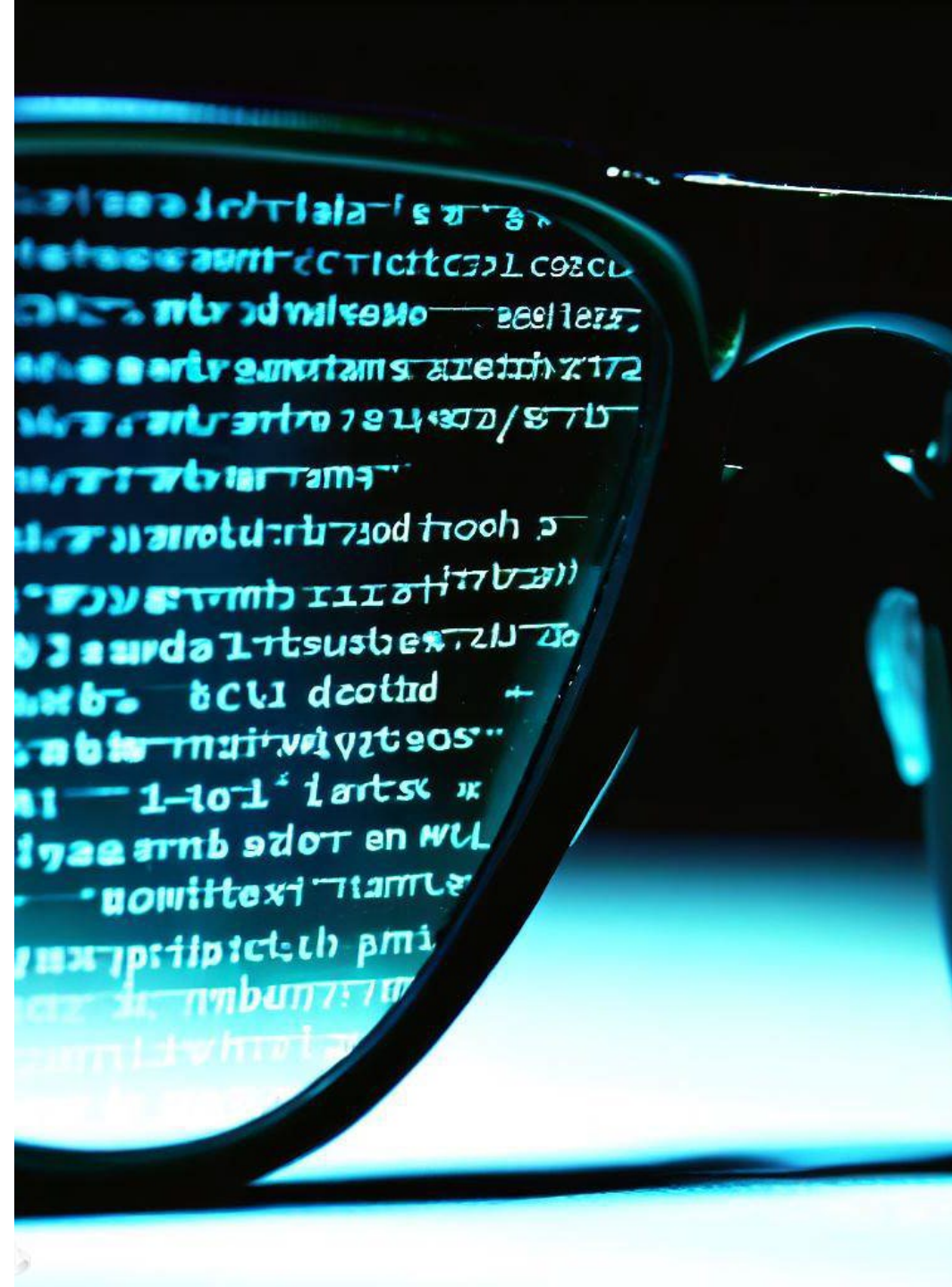
Data Dependency Conflict

- Consider a **4-stage pipeline** (FDES).
- Actual operation of each instruction e.g. ADD, SUB, is done during the Execute (E) stage.
- The resultant value of the execution is transferred to the destination during the Store (S) stage.
- In the example below, the **old value** (0x0) is still in **R2** when **SUB** instruction is in **Execute** (E) stage, so R3 will have the wrong value (0x5) instead of the correct value (0x3).



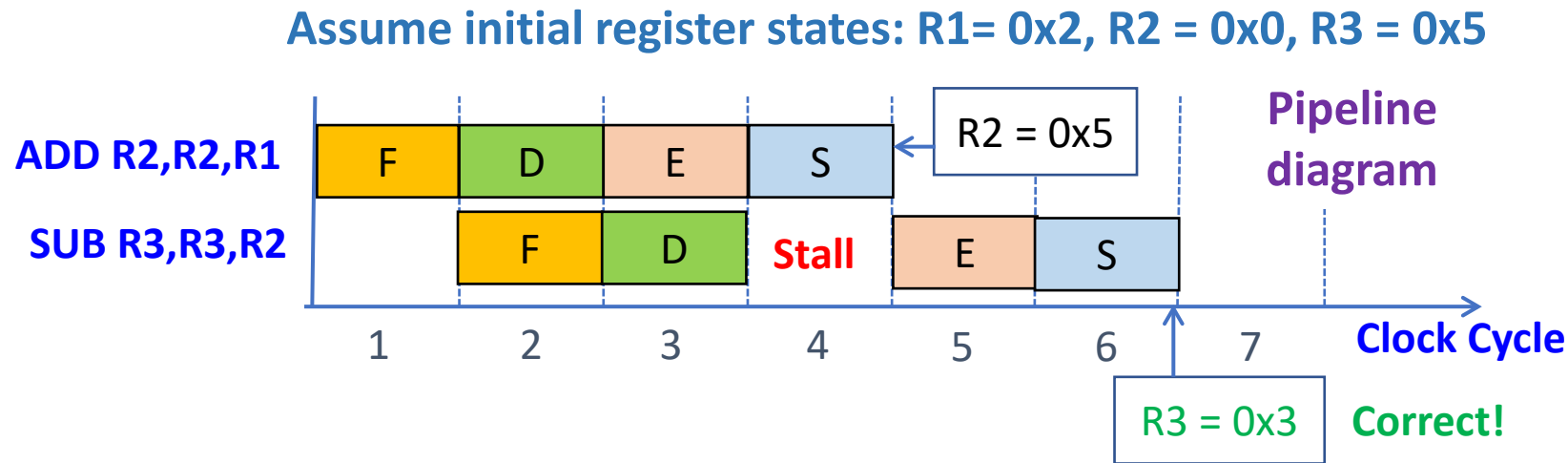
Data Dependency Conflict

- For the example in the previous slide, it will not be an issue if the processor has a simple CPU architecture where instruction execution is sequential. The current instruction will start its execution only after the previous instruction has complete. So the most updated copy of the memory/register value is always available to the current instruction.
- Due to the overlapping nature between instructions within a pipeline, there will be instances where a data required for the proper execution of the current instruction has not been updated yet.
- This is known as data dependency issue/conflict. It arises when the source operand of the current instruction is also the destination operand of a prior instruction.
- The exact separation between the two instructions in order for the issue to occur depends on the pipeline structure and design.

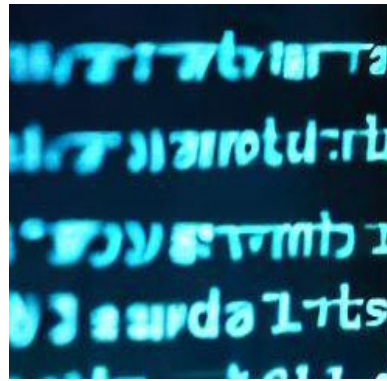


Resolving Data Conflict – Stall the Pipeline

- **Hardware circuitry** can be used to detect data dependency between instructions
 - Compare destination identifier in the Execute Stage with source(s) in the Decode stage.

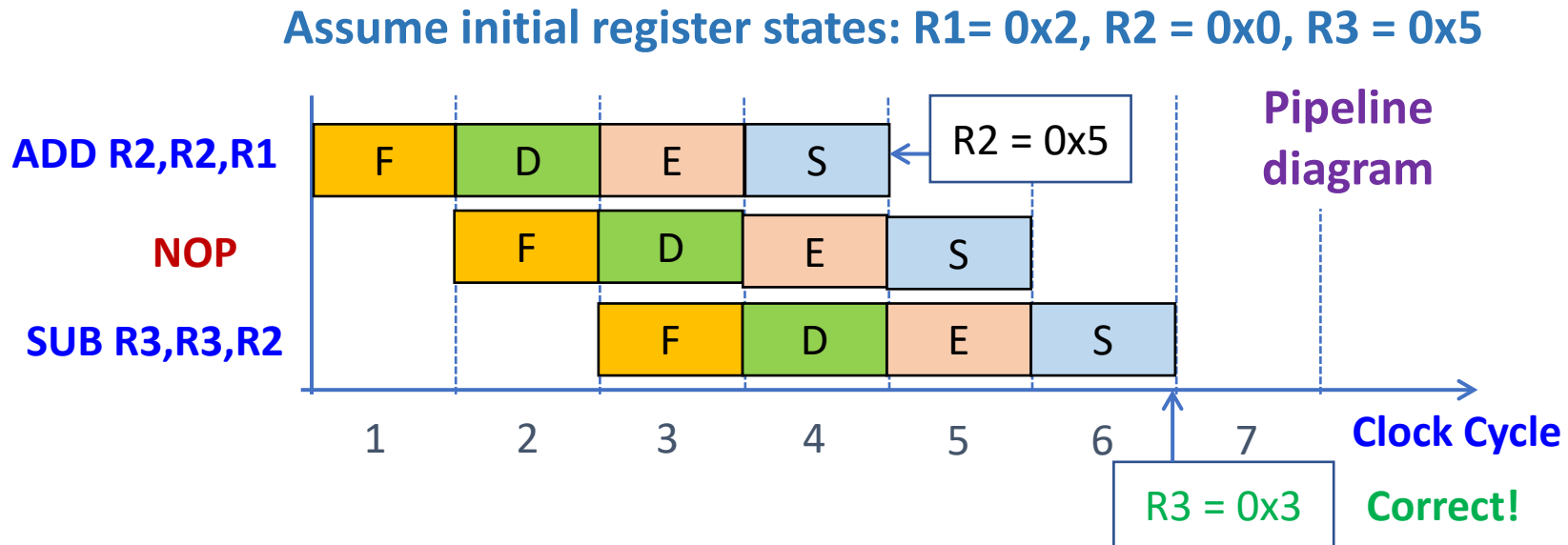


- If data dependency is detected (i.e. **R2 matches**), allow **ADD** to continue normally, but **stall** the **Decode** stage of **SUB**.
- After ADD completes, SUB is allowed to resume.



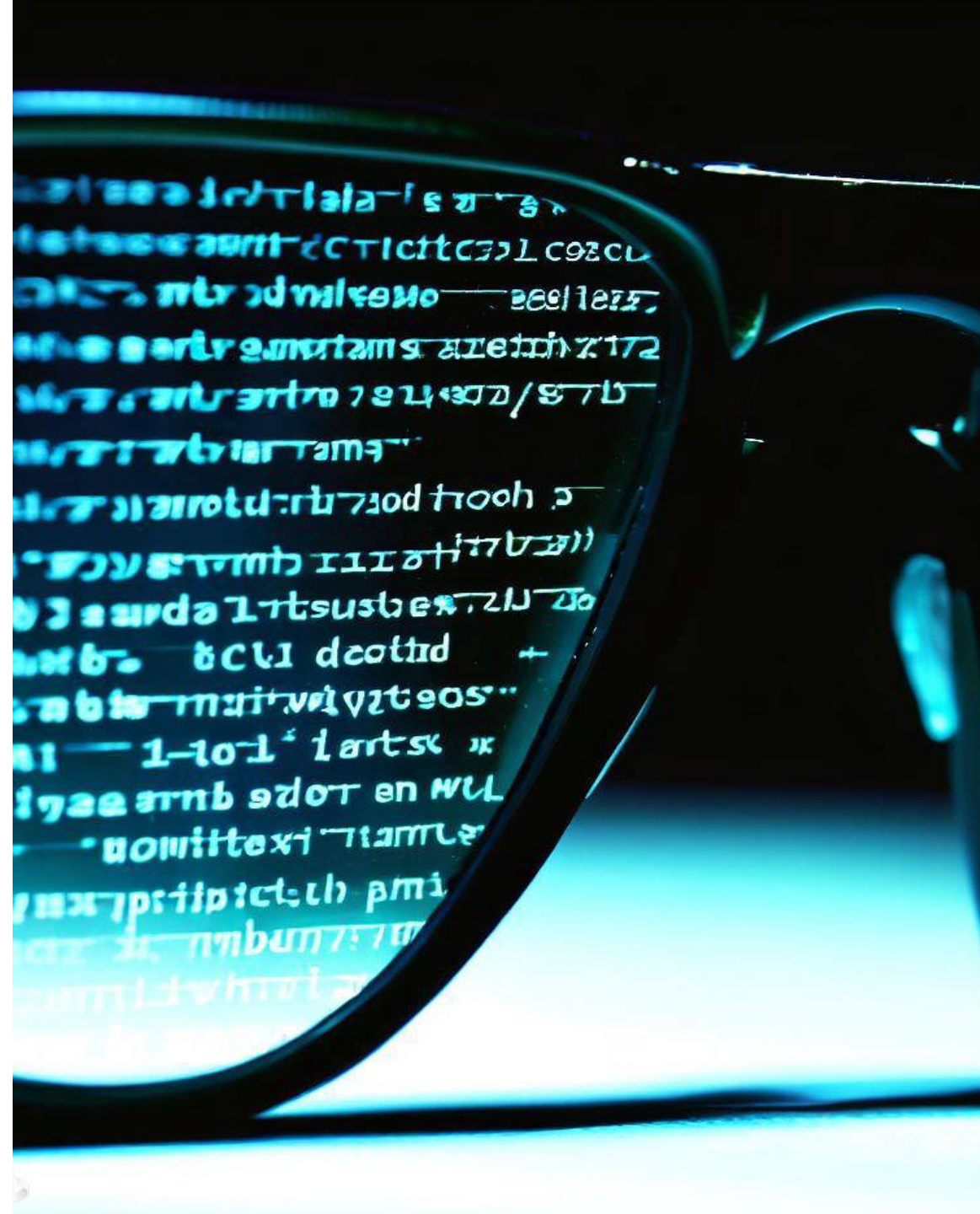
Resolving Data Conflict – Insert NOP Instructions

- **Compiler** can analyze and **insert redundant instructions** to reduce data conflict
 - Data dependencies are evident in instructions during compilation
 - Compiler inserts explicit **NOP** (No Operation) instructions **between instructions with data dependencies**
- Delay ensures new value is available in register but causes total execution time to increase



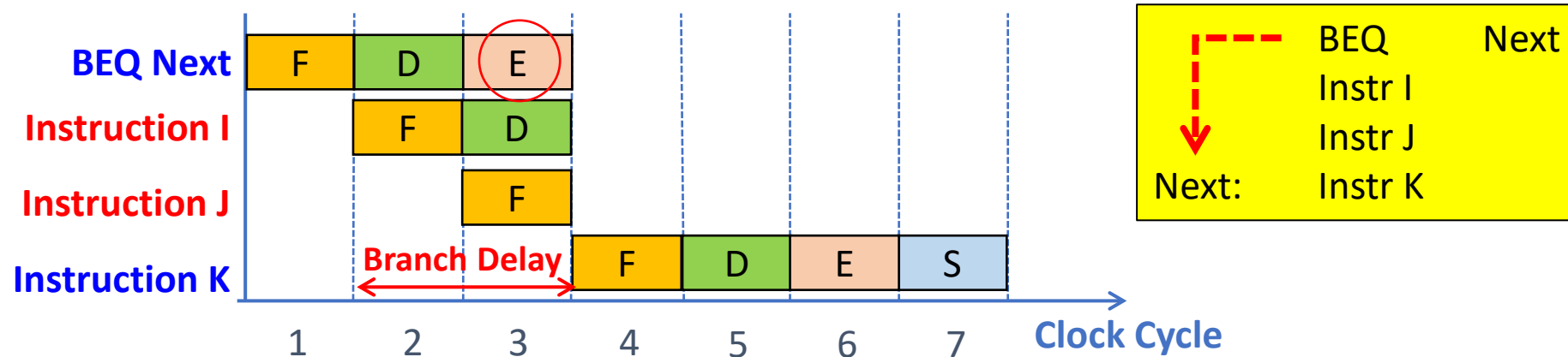
Branch Instruction

- Branch instruction usually need to perform two operations
 - Evaluate condition to determine if branch should be taken/not taken
 - If branch is taken, calculate branch target using adder in ALU
- Both operations require an ALU to perform some computation and processing so a natural stage to do this is in the Execute (E) Stage of the pipeline.
- The processor would need to flush the pipeline to reload correct instructions according to the branch decision. Flushing of pipeline is equated to wastage in cycles for instruction execution.
- The number of cycles wasted/lost is known as Branch Delay.



Branch Delay

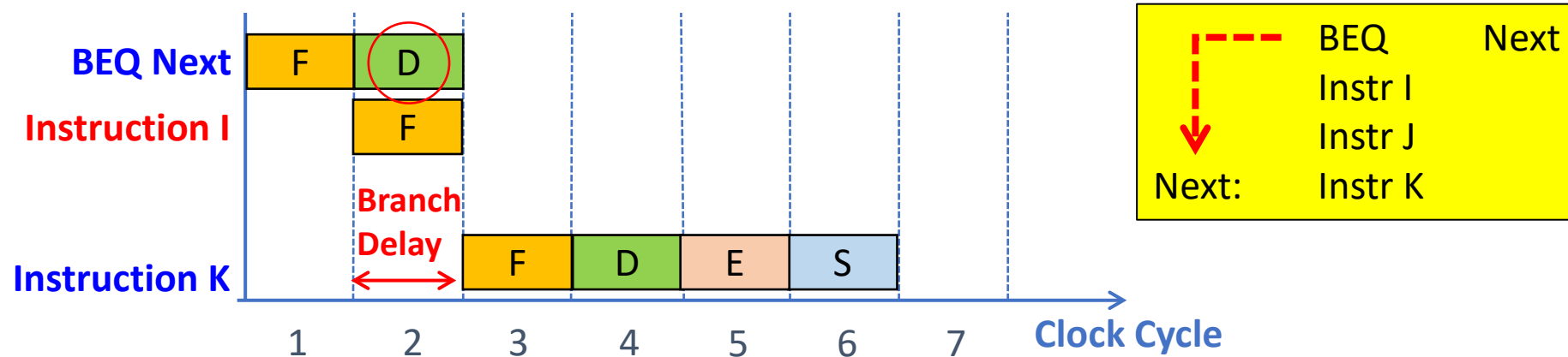
- Branch statements in pipelining can lead to **Branch Delay**.



- The **branch target** is only known after the **Execute** stage, but by this time, **Instructions I and J** have already been **fetched**.
- Instructions I and J will be **discarded**, resulting in **two-cycle branch delay**.
- The two slots that are discarded is known as the **Delay Slots**
- Instructions I and J are known as the **Delay Slot Instructions**

Reducing Branch Delay

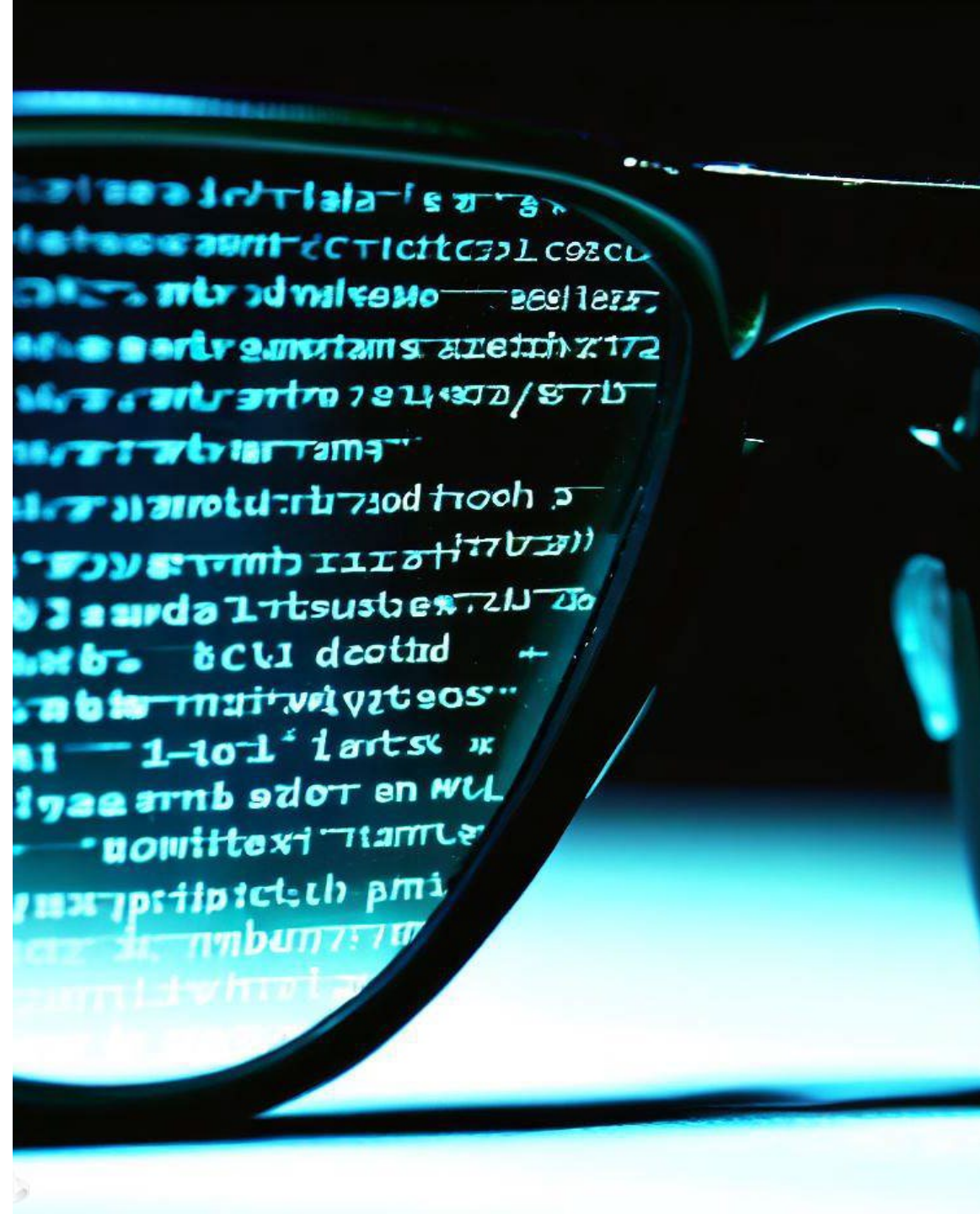
- Branch delay can be **reduced** by making the branch decision and calculating the branch target **earlier at the Decode stage**.
- An **additional adder** is introduced to be used in the Decode stage to enable earlier calculation of branch target.



- After the **Decode stage**, the branch decision and the branch target is known.
- Hence if **branch is taken**, only **Instruction I** needs to be **discarded**.
- Branch delay is **reduced to one cycle**.

Delayed Branching

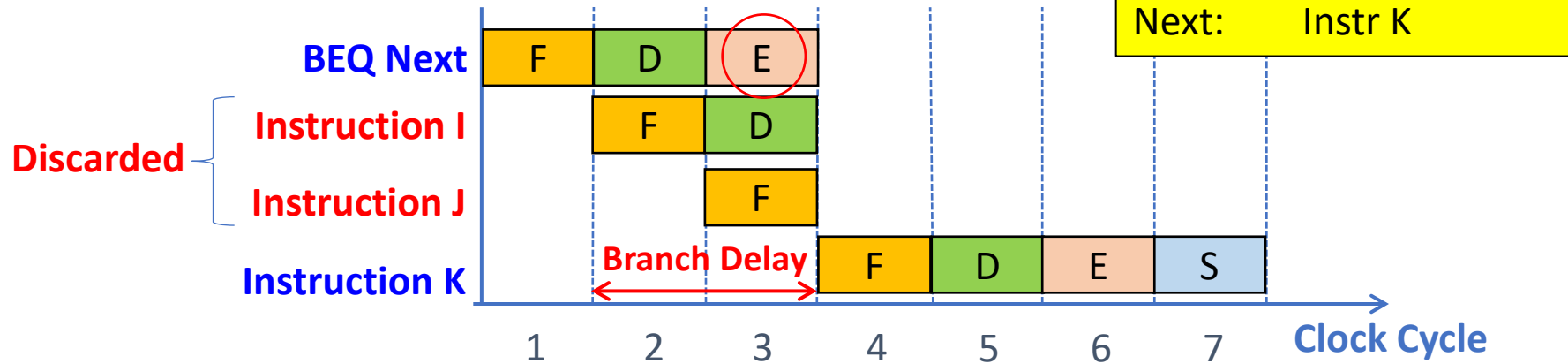
- In the previous implementation, the instruction(s) immediately following a branch is always fetched, regardless of the branch decision.
- If branch is taken, these instruction(s) will be discarded resulting in branch delay.
- Delayed Branching (to be introduced next) is a method that ensures no instructions are discarded after the branch. That means that delay slot instructions are always executed.



Delayed Branching: Disabled

- For simplification sake, we can view Delayed branching as a feature/choice in the processor
- If Delayed branching is disabled, the processor pipeline will discard the delay slot instructions if the branch is True to preserve the correctness of the program logic, at the expense of cycle wastage.

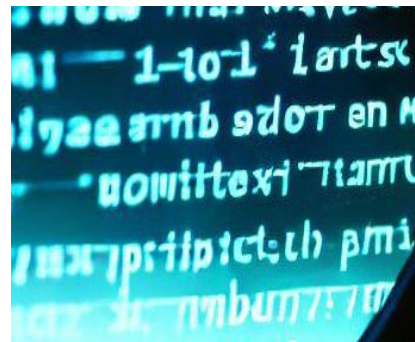
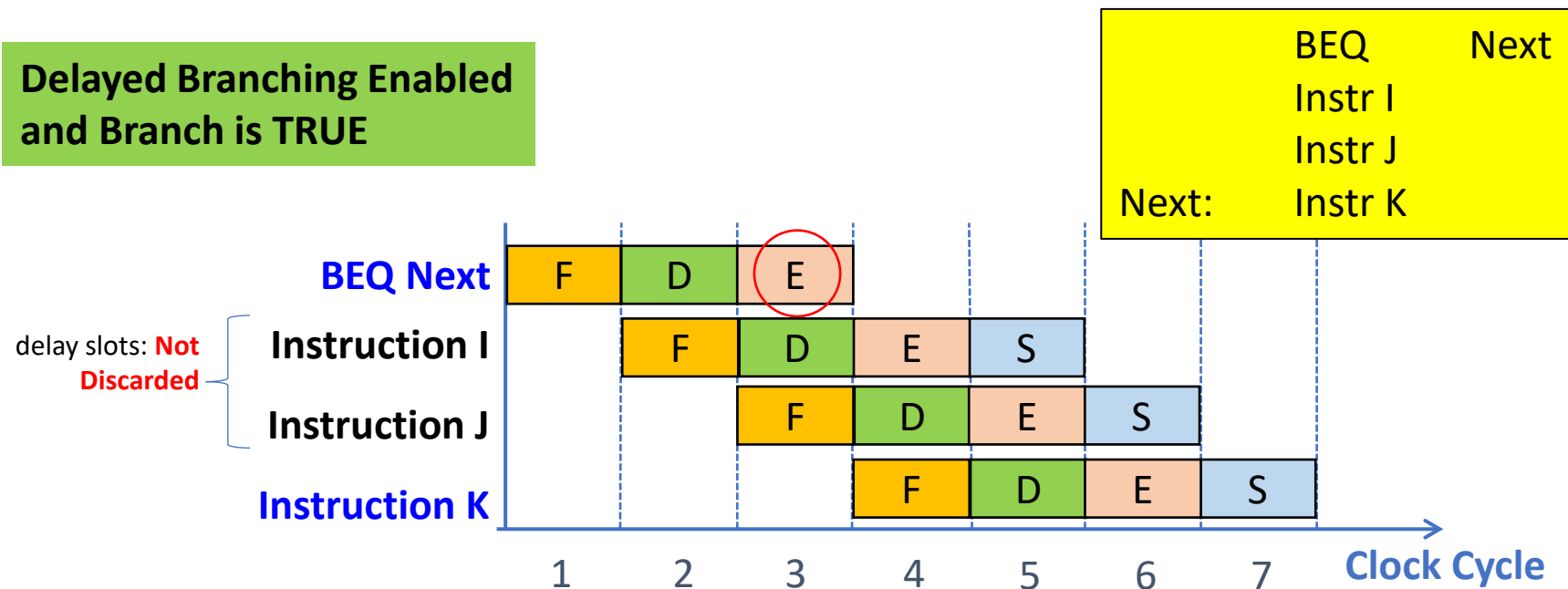
Delayed Branching Disabled
and Branch is TRUE



Delayed Branching: Enabled

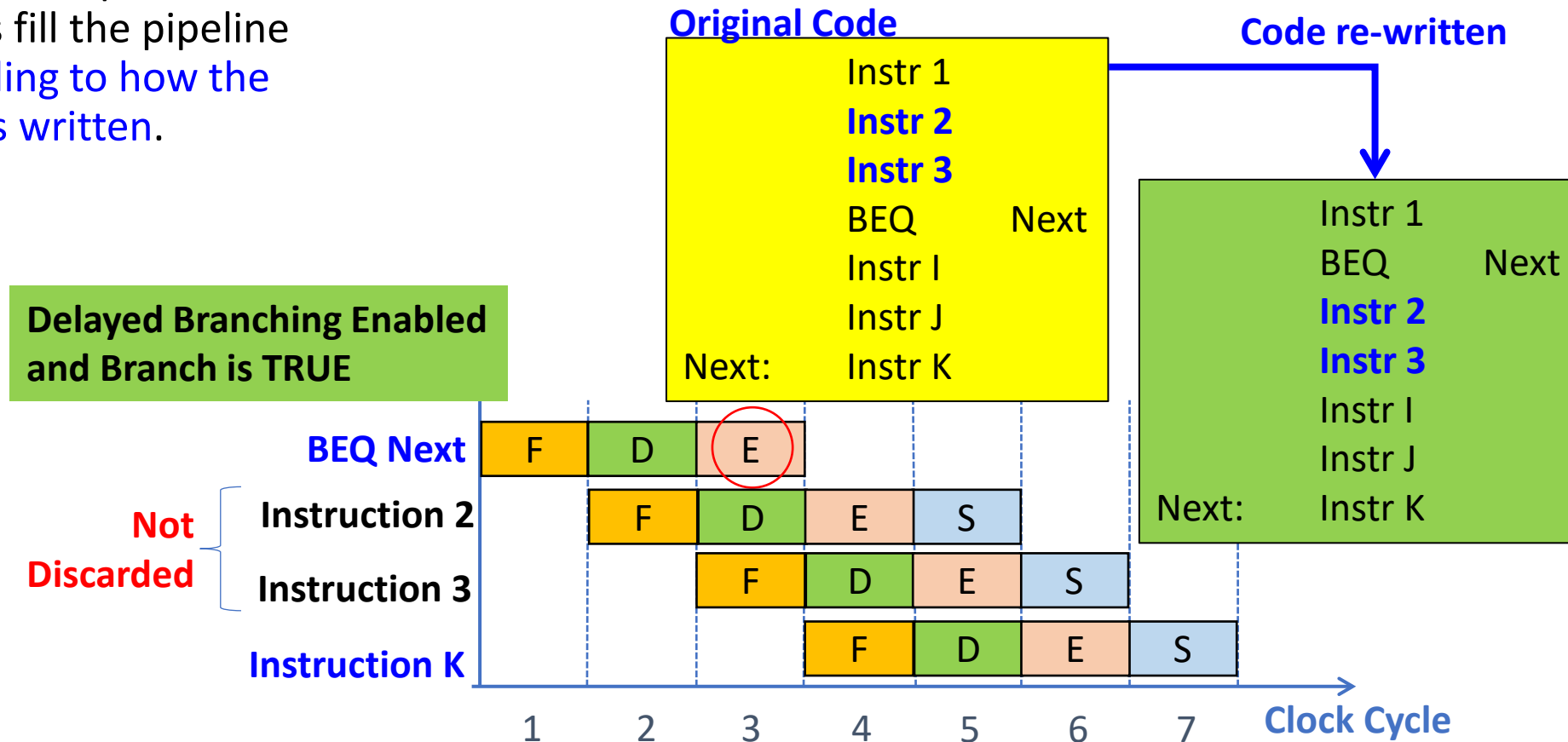
- If the Delayed branching is **enabled**, the processor will **execute** the **delay slots instructions** regardless of true or false branch decision so **no cycles are wasted**.
- However, **the program logic would be wrong** if instruction **I** and **J** are executed, when the branch is True.
- So, **some other instructions** (other than I and J) need to fill the delay slots.

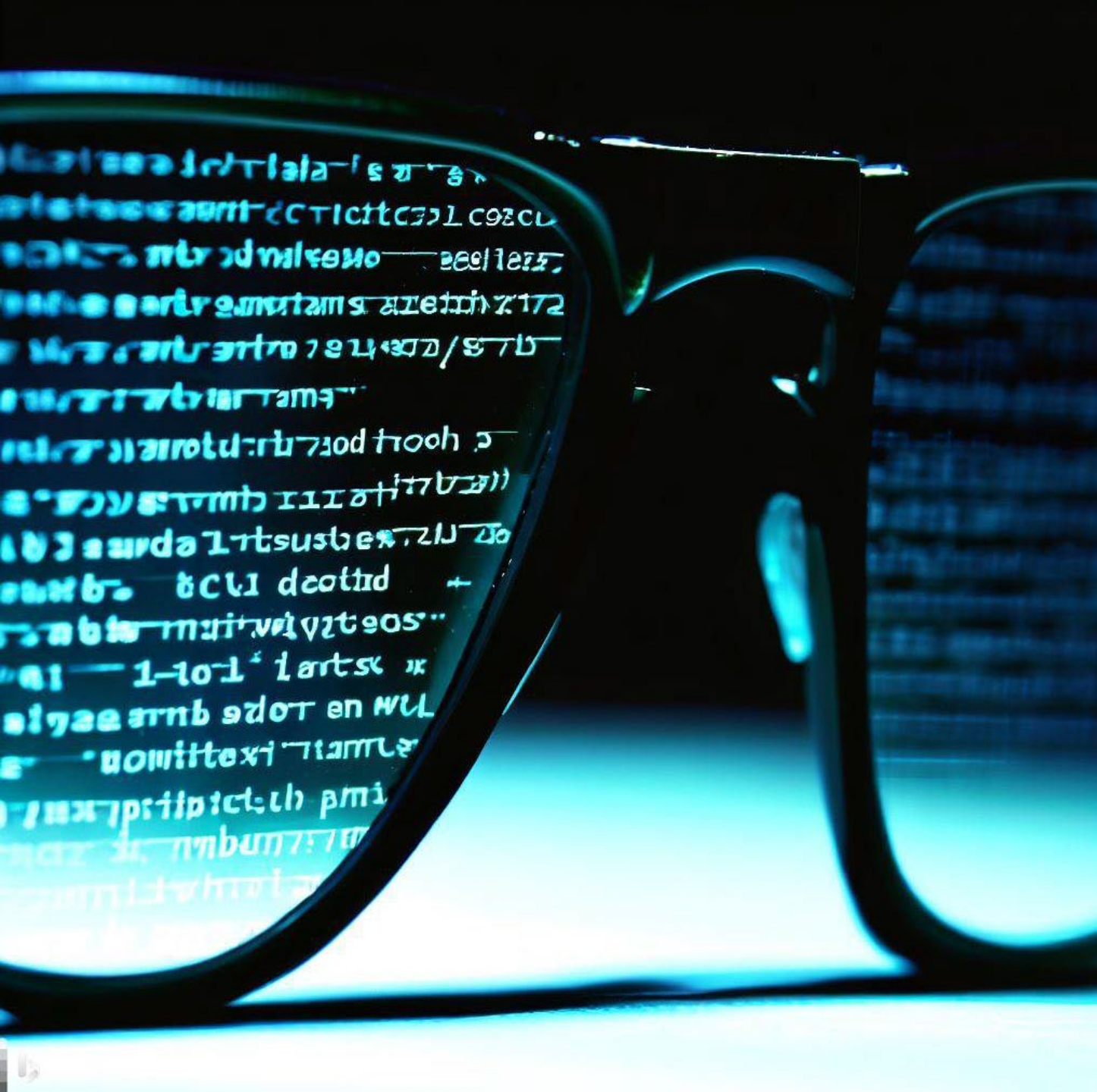
**Delayed Branching Enabled
and Branch is TRUE**



Delayed Branching

- The user or compiler needs to fill the delay slots with Independent instructions, by **changing the program**.
- For example, if **instructions 2 and 3 are independent instructions**, they can be made to occupy the delay slots by re-writing the program code.
- Note that a processor always fill the pipeline according to how the code is written.





Delayed Branching

- User or Compiler can schedule independent instructions to be filled in the delay slots after the branch.
- If such independent instruction exists, they will always be executed, leading to zero branch delay, since no cycles are wasted.
- If an independent instruction cannot be found or if there are insufficient number of independent instructions to fill the delay slots, NOP instruction(s) should be used to populate the delay slots to preserve the correctness of the original program logics.

Delay slot Instructions

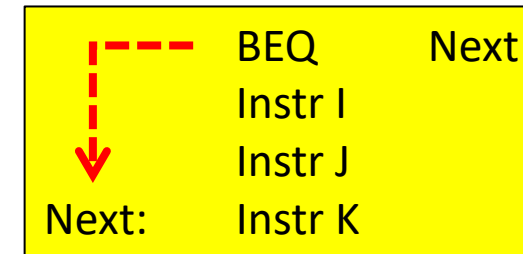
- Needs to be Independent instructions which **does not play a part in the branch decision making process.**
- Another requirement for delay slot instructions is that they will **always be executed in the original program flow**, regardless of whether the branch is taken or not.
- There are a few **general rules of thumb**
- It should be some instructions that are **earlier sequence wise** in the original program, compared to the branch instruction. Any instruction that occurs later than the branch instruction in the original program would be executed or not executed depending on the branch decision.
- Its operation **should not have effect on status of any registers that would in turn affect the branch decision.** e.g. if a BEQ is used, the independent instruction should not affect the Z flag which BEQ instruction used for decision making.
- This is because **delay slot instructions gets fully executed only after the branch decision had been made**, which means delay slots instruction had **no effect on the branch decision.**
- If the Branch is **part of a loop**, delay slot instructions needs to be **sourced from instructions within the loop** as delay slot instruction will go through the same number of iterations as well.



Dynamic Branch Prediction:

another way to mitigate branch delay

- Hardware circuitry to guess outcome of a conditional branch
 - **Branch history table is implemented to store the predicted target addresses of branch instructions in the program**
- If prediction is **correct**
 - **Continue normal execution – no wasted cycles**
- If prediction is **incorrect**
 - **Flush instructions that were incorrectly fetched – wasted cycles**
 - **Update prediction bit and target address for future use**



Address of Branch Instruction	Predicted Target address	Prediction Result (T/F)
Address of "BEQ Next"	Next	T



Connecting to the Real World

- ARM Cortex M3/M4 Processor
 - 3-stage pipeline. Instruction Fetch, Instruction Decode and Instruction Execute)
- Branch speculation.
 - When a branch instruction is encountered, the decode stage also includes a speculative instruction fetch that could lead to faster execution.
 - The processor fetches the branch destination instruction during the decode stage itself.
 - During the execute stage, the branch is resolved and it is known which instruction is to be executed next.
 - If the branch is not taken, the next sequential instruction is already available.
 - If the branch is taken, the branch instruction is made available at the same time as the decision is made.



No part of this material shall be filmed, recorded, downloaded, reproduced, distributed, republished, or transmitted in any form or by any means without written approval from Nanyang Technological University.